Switch 2 Controller Emulation for yuzu-family Emulators:
Gyro, HD Rumble, and Amiibo/NFC

1. Native Switch controller behavior vs. general PC controller support

The app's existing general-purpose gyro passthrough and PC controller support is a very valid and important part of the project. Joy-Con 2 controllers should be usable across Windows applications, Steam, SDL-based software, native PC games, gyro-as-mouse setups, Ryujinx-style input paths, and other tools.

The narrower topic here is yuzu-family emulator compatibility: Eden, Citron, suyu, and related forks run native Nintendo Switch games. Those games were designed around Nintendo's own controller stack: Joy-Con / Pro Controller identity, IMU reports, calibration behavior, report timing, HD Rumble semantics, and NFC/Amiibo behavior.

That makes this use case different from using Joy-Con 2 controllers as general-purpose PC controllers.

- **SDL / Generic Gyro Passthrough Path** - broad PC compatibility, Steam/SDL/Ryujinx-style input, native PC games, and user-facing gyro tuning.
- **Native Nintendo HID Compatibility Path** - strict Switch 1 Joy-Con / Pro Controller behavior at the HID/report/protocol level for Eden/yuzu-family direct Joy-Con Driver mode.

Ryujinx/Ryubing can support motion controls natively in many cases, meaning motion can work through the emulator's input configuration and input backend without necessarily using a CemuHook motion server. But this is still different from Eden/yuzu's direct Joy-Con Driver model. Ryujinx-style native motion is primarily an input-backend / SDL-style path, while Eden/yuzu's direct Joy-Con Driver is a Nintendo HID protocol path with device identity, calibration, IMU setup, active reports, native rumble reports, and NFC-related protocol handling.

So passing Ryujinx/Ryubing motion tests can show that the app's general-purpose gyro output is useful. Passing Eden/yuzu direct Joy-Con Driver tests shows something stricter: that the virtual controller behaves like a Switch 1 Joy-Con at the HID/report/protocol level.

Sources: Eden official website, TommyWabg Switch 2 Controllers repository, TommyWabg releases, Ryujinx/Ryubing setup guide, EmuDeck Ryujinx guide, Ryujinx SDL motion issue context.

2. What the yuzu-family direct Joy-Con Driver does

Although this feature is usually called the "Joy-Con Driver", the name is somewhat narrow. In yuzu-family source code, the same native Nintendo HID path also recognizes the Switch Pro Controller.

Relevant IDs include Nintendo Vendor ID `0x057e`, Joy-Con L Product ID `0x2006`, Joy-Con R Product ID `0x2007`, and Switch Pro Controller Product ID `0x2009`. After detection, the driver opens the device directly with `SDL_hid_open(...)` and uses non-blocking HID I/O.

So the important distinction is not simply "Joy-Con versus non-Joy-Con". The important distinction is native Nintendo controller HID path vs. generic PC controller abstraction.

This direct path is special because it can preserve Switch-like controller behavior much more closely than generic input paths. It is where native-style IMU reports, controller calibration, HD Rumble

semantics, NFC/Amiibo protocol handling, and Joy-Con / Pro Controller timing can exist in their original-like form.

Sources: Joy-Con Driver source, raw source, PCGamingWiki Joy-Con overview.

IMU, calibration, and timing

During initialization, the Joy-Con Driver sets up protocol modules such as Calibration, Generic functions, IR, NFC, Ring-Con, and Rumble. It also sets important defaults: `vibration_enabled = true`, `motion_enabled = true`, gyro sensitivity `DPS2000`, gyro performance `HZ833`, accelerometer sensitivity `G8`, and accelerometer performance `HZ100`.

If motion is enabled, the driver calls `EnableImu(true)`, sets the IMU configuration, and usually switches the controller into Active Mode. The driver also reads stick calibration and IMU calibration, and those calibration values are then used by the Joy-Con poller.

This matters because Eden/yuzu does not just consume "some gyro value". It expects Switch 1-style IMU data that matches the selected range, calibration, axis layout, report format, and timing.

When motion is active, the driver reads reports through `SDL_hid_read_timeout(...)`. Reports are handled as `STANDARD_FULL_60HZ`, `NFC_IR_MODE_60HZ`, or `SIMPLE_HID_MODE`, and the driver calculates a smoothed `delta_time` from report timing. A high-frequency gyro pipeline is useful for many PC applications, but strict Joy-Con HID emulation still has to output what the emulator expects from a real Switch 1 controller.

Sources: Joy-Con Driver source, raw source, JoyShockLibrary-plus-HDRumble.

Why CemuHook/DSU, BetterJoy, and Ryujinx-style input paths are different

CemuHook/DSU is an external motion-source route. It uses UDP and allows external software to send button and motion data to an emulator.

BetterJoy is a classic external compatibility approach. Its own README describes it as allowing Switch Pro Controller, Joy-Cons, and Switch SNES controller to be used with Cemu using CemuHook, Citra, Dolphin, yuzu, and system-wide with generic XInput support. In other words, BetterJoy is very useful, but it is primarily a PC/emulator compatibility layer: it exposes Nintendo controllers through CemuHook and/or generic PC controller interfaces.

This explains why using BetterJoy as a reference for Switch 1 emulation is useful but not complete for Eden/yuzu's direct Joy-Con Driver target. BetterJoy can help understand controller data, gyro, and PC compatibility, but Eden/yuzu's direct Joy-Con Driver expects a Nintendo HID device at the report/protocol level: identity, subcommands, calibration, IMU setup, active reports, native rumble reports, and NFC-related behavior.

Ryujinx/Ryubing input is also a different target. Motion can work natively through its input configuration/backend for supported controllers, and guides also describe optional CemuHook-compatible motion paths. This is useful, but it is not the same as Eden/yuzu's direct Joy-Con Driver, where the emulator expects a Nintendo-style HID device.

For Eden/yuzu's direct Joy-Con Driver path, the target is closer to a virtual Switch 1 Joy-Con at the HID/report/protocol level.

Sources: CemuHook / DSU protocol documentation, BetterJoy repository, Fossephate JoyCon-Driver, Ryujinx readme mirror, Ryujinx/Ryubing setup guide, EmuDeck Ryujinx guide, Joy-Con Driver source.

Origin of the native Joy-Con path

The direct Joy-Con Driver comes from yuzu development and was inherited by later forks such as Eden, Citron, suyu, and others. The source still contains yuzu copyright information, for example `Copyright 2022 yuzu Emulator Project`.

The Joy-Con rumble code references several sources and inspirations, including `dkms-hid-nintendo`, CTCaer's Joy-Con Toolkit, and dekuNukem's Nintendo Switch reverse-engineering documentation. In short: the direct Joy-Con Driver is a yuzu-family low-level Nintendo HID implementation based on reverse engineering, Linux/Nintendo HID knowledge, and emulator-side controller work.

Sources: Joy-Con Driver source, raw source, Eden/yuzu-family rumble header, suyu/yuzu Joy-Con driver history RSS, suyu/yuzu Joy-Con generic functions commit.

3. Why gyro likely needs two output profiles

The app likely benefits from two separate gyro output profiles:

- **SDL / Generic Gyro Passthrough Path**
- **Native Nintendo HID Compatibility Path**

The general-purpose path is valuable for Steam, SDL, Ryujinx-style input, native PC games, gyro-as-mouse, and other PC use cases. It can reasonably include user-facing features such as sensitivity, calibration, magnetometer calibration, 9-axis assistance, horizon lock, soft deadzone, anti-shake behavior, and high-frequency gyro handoff.

The strict Switch 1 Joy-Con HID path is different. Its goal is not just a useful PC gyro signal. Its goal is to produce something that Eden/yuzu sees as a real Switch 1 Joy-Con.

That likely means Switch 1-compatible raw IMU values, Switch 1-compatible gyro and accelerometer scale, `DPS2000`-compatible gyro behavior, correct axis order and signs, correct IMU calibration responses, correct active report format, stable Joy-Con-like timing, and no generic mouse/SDL post-processing in the final HID report unless explicitly verified.

The v0.9.4 behavior is useful evidence for this separation: the gyro scaling change appeared to move the native Joy-Con Driver path closer while making SDL/generic gyro too weak. That suggests this kind of conversion may belong in the strict Switch 1 HID profile rather than in the shared gyro path.

A possible strict emulator path would look like:

> Joy-Con 2 sensors -> minimal hardware correction -> Switch 1 IMU scale conversion -> Joy-Con 1-style calibration-compatible HID reports -> Eden/yuzu native Joy-Con Driver

That keeps the broad PC gyro pipeline intact while adding an emulator-specific compatibility profile.

Sources: TommyWabg Switch 2 Controllers repository, TommyWabg releases, TommyWabg tags, Joy-Con Driver source, raw source, JoyShockLibrary-plus-HDRumble, Ryujinx/Ryubing setup guide.

4. Why rumble likely needs two translation paths

Gyro is input: **Joy-Con 2 -> app -> Eden/yuzu**.

Rumble is output: **Eden/yuzu -> app -> Joy-Con 2**.

So the rumble equivalent is not exactly "two HD Rumble outputs". It is better described as two rumble receive/translation paths:

- **SDL / Generic Rumble Path**

- **Native Nintendo Rumble Translation Path**

SDL / generic rumble

SDL/generic rumble is useful for normal PC controller compatibility. It may involve generic motor strength, Xbox-style dual-motor behavior, simplified low/high motor behavior, strength sliders, frequency sliders, and broad compatibility with PC games and SDL applications.

However, it may not preserve all Nintendo HD Rumble information. If Eden/yuzu is not using the direct Joy-Con Driver, rumble may be reduced to a generic abstraction.

This also explains why short repeated vibration events can disappear in SDL/generic rumble paths. SDL's gamepad rumble API is a duration/intensity effect model, and SDL3 documentation states that each call to `SDL_RumbleGamepad` cancels any previous rumble effect. If several short effects arrive quickly, such as multiple coin pickups, a generic rumble path may overwrite or merge earlier effects before they are felt.

So SDL/generic rumble is useful as a compatibility fallback, but it should not be treated as the main target for preserving Switch-style HD Rumble.

Sources: SDL3 SDL_RumbleGamepad documentation, TommyWabg app repository, yuzu Progress Report October 2020, PCGamingWiki Joy-Con overview.

Native Nintendo Joy-Con rumble translation

When Eden/yuzu's direct Joy-Con Driver is enabled, the emulator sends native Nintendo Joy-Con rumble reports. Those reports contain more specific information, including low frequency, high frequency, low amplitude, and high amplitude.

The app's Switch-style rumble path can aim to preserve Nintendo rumble semantics as much as possible:

> Native Nintendo rumble report -> parse frequency/amplitude information -> translate to Switch 2 controller haptics -> send to physical Joy-Con 2

This is the relevant target for HD Rumble in native Switch emulation. The app's existing Switch-style rumble mode already sounds conceptually aligned with this, because it is described as mimicking native Switch HD Rumble / LRA experience and routing raw frequency data more directly to the controller.

Sources: Eden/yuzu-family rumble header, Eden mirror rumble implementation, TommyWabg app repository, yuzu Progress Report October 2020, JoyShockLibrary-plus-HDRumble.

Accurate Vibrations ON/OFF

"Accurate Vibrations" is a yuzu vibration option. The yuzu October 2020 progress report explains that reduced vibration rates helped avoid stutters on some setups, but could also cause missing or incomplete effects. That is why yuzu introduced the Accurate Vibrations toggle.

- **Accurate Vibrations ON:** fuller vibration implementation, more detailed/frequent updates.
- **Accurate Vibrations OFF:** vibration updates are reduced through heuristics.

When Accurate Vibrations is disabled, yuzu limits the effective vibration rate to 100 vibrations per second, excluding zero-amplitude states.

This explains an important observation:

- Real Switch 1 Joy-Cons + native Joy-Con Driver + Accurate Vibrations ON can produce the better HD Rumble experience because the full native stream reaches hardware that understands it directly.

- Joy-Con 2 + app + native Joy-Con Driver + Accurate Vibrations OFF can currently produce the better result because the app has to translate the stream, and the reduced stream is easier to handle.
- Joy-Con Driver OFF / SDL generic rumble does not provide the same native Nintendo rumble information, so Accurate Vibrations ON/OFF cannot restore real HD Rumble in the generic path.

So Direct Joy-Con Driver ON decides whether native Nintendo rumble information exists at all. Accurate Vibrations decides how complete, dense, or demanding that native rumble stream is.

Sources: yuzu Progress Report October 2020, Joy-Con Driver source, raw source, Eden mirror rumble implementation.

5. Why Amiibo/NFC belongs to the native Nintendo path

Amiibo/NFC is another example of why the native Nintendo controller path matters. It is not a generic PC controller feature; it belongs to Nintendo's native controller and system-service behavior.

In yuzu-family source code, NFC is part of the Joy-Con protocol area. The Joy-Con Driver initializes an `NfcProtocol`, and yuzu-family code contains NFC-related functions such as `EnableNfc()` and NFC polling. There are also historical yuzu-family fixes specifically mentioning NFC detection for Joy-Cons and Amiibo support for the Pro Controller.

Eden also documents file-based `.bin` Amiibo loading as an emulator-side workflow. That file-based workflow is useful and can coexist with native controller-side NFC behavior. But the presence of Joy-Con NFC protocol code and historical Pro Controller Amiibo support shows that Amiibo/NFC also belongs conceptually to the native Nintendo controller path.

For Switch 2 Joy-Con emulation, NFC/Amiibo could become a future compatibility target after gyro and HD Rumble if Joy-Con 2 exposes NFC data to Windows in a usable way. File-based Amiibo loading could remain available as a fallback, while physical NFC passthrough would belong to the stricter native Nintendo controller compatibility path.

Sources: Eden Amiibo guide, Joy-Con Driver source, yuzu-family NFC header, yuzu-family NFC implementation, yuzu-family NFC header reference, Pro Controller Amiibo support commit, Amiibo writing support commit.

6. Practical compatibility checklist

Gyro

For yuzu-family native Joy-Con Driver mode, the strict Switch 1 Joy-Con HID profile should focus on Switch 1-compatible raw IMU behavior, correct `DPS2000` gyro scale, correct accelerometer scale, correct axes/signs, calibration-compatible output, Joy-Con-like timing, native active reports, and no generic mouse/SDL post-processing in the final HID report unless explicitly verified.

Useful A/B test: real Switch 1 Joy-Con + direct Joy-Con Driver vs. Joy-Con 2 through the app + Switch 1 emulation mode + direct Joy-Con Driver. Compare raw gyro, raw accelerometer, calibrated values, idle noise, bias, axes, signs, report interval, packet counter, slow movement, fast flicks, and behavior after sudden stops.

Sources: Joy-Con Driver source, raw source, TommyWabg releases.

Rumble

For rumble, the app may benefit from two separate translation paths: SDL/generic rumble for broad PC compatibility and native Nintendo Joy-Con rumble for Eden/yuzu-family direct Joy-Con Driver mode.

Useful tests: real Joy-Con 1 + native Joy-Con Driver + Accurate ON as original-style HD Rumble reference; Joy-Con 2 + app + native Joy-Con Driver + Accurate OFF as currently working translation

path; Joy-Con 2 + app + native Joy-Con Driver + Accurate ON as full native-rumble translation stress test; Joy-Con 2 + app + SDL/generic rumble as fallback comparison.

Also test short repeated rumble events, such as collecting several coins or triggering several small pickup/UI events in quick succession. If SDL/generic rumble drops early events and only plays a later one, that suggests event merging, throttling, overwriting, or queue loss in the generic rumble path.

Sources: Eden/yuzu-family rumble header, Eden mirror rumble implementation, yuzu Progress Report October 2020, SDL3 SDL_RumbleGamepad documentation, TommyWabg app repository.

Amiibo/NFC

Amiibo/NFC does not have to be solved before gyro and HD Rumble. But it is a useful future direction because it belongs to the same native Nintendo controller experience.

Possible future research areas: whether Joy-Con 2 exposes NFC data to Windows in a usable way; whether that data can be mapped into the yuzu-family Joy-Con NFC protocol; how yuzu-family emulators request NFC state through `joycon_protocol/nfc`; how higher-level NFP/Amiibo services interact with controller NFC data; and whether physical Amiibo passthrough can coexist with file-based `.bin` Amiibo loading.

Sources: Eden Amiibo guide, yuzu-family NFC header, yuzu-family NFC implementation, Pro Controller Amiibo support commit.

Implementation reference

Since yuzu-family emulators already support real Switch 1 Joy-Cons and Pro Controllers through the native Nintendo controller path, the cleaner compatibility goal is likely app-side:

> Make the virtual Switch 1 Joy-Con close enough that Eden/yuzu does not need special handling.

Eden/yuzu source is useful as a specification reference: VID/PID, subcommands, calibration, IMU mode, rumble reports, timing/queues, and NFC/Amiibo requests.

Ryujinx/Ryubing behavior can still be useful for general PC input testing, but it should not be treated as equivalent to Eden/yuzu's direct Joy-Con Driver path. Passing Ryujinx-style SDL/backend motion tests does not necessarily prove that the virtual controller is strict Switch 1 Joy-Con HID-compatible.

Sources: Joy-Con Driver source, raw source, Eden/yuzu-family rumble header, Eden mirror rumble implementation, yuzu-family NFC header, Ryujinx/Ryubing setup guide, EmuDeck Ryujinx guide.

Short summary

General PC controller support and native Switch controller emulation are different targets.

The existing gyro passthrough and broad PC support are valuable for Steam, SDL, Ryujinx-style input paths, native PC games, gyro mouse, and many other applications.

Eden/yuzu-family emulators using the direct Joy-Con Driver are a stricter special case because they run native Switch games and expect Nintendo-style controller behavior at the HID/report/protocol level.

For gyro, the app likely benefits from two separate output profiles: **SDL / Generic Gyro Passthrough Path** and **Native Nintendo HID Compatibility Path**.

For rumble, the app likely benefits from two separate receive/translation paths: **SDL / Generic Rumble Path** and **Native Nintendo Rumble Translation Path**.

For Amiibo/NFC, the app may eventually benefit from an additional native-controller compatibility layer if Joy-Con 2 NFC data is accessible from Windows and can be mapped into yuzu-family NFC expectations.

The likely path forward is not one global gyro or rumble setting, but clear separation between broad PC compatibility and strict native Nintendo controller compatibility.

The final target for Eden/yuzu-family native controller mode is:

> **The virtual Switch 1 Joy-Con device should behave closely enough at the HID report, calibration, timing, gyro, rumble-report, and eventually NFC/Amiibo level that the emulator's direct Nintendo controller path cannot distinguish it from a real Switch 1 Joy-Con or Pro Controller where applicable.**